

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Обработка изображений в интеллектуальных
системах»

Тема: «Обучение классификаторов средствами библиотеки
PyTorch»

Выполнил:

Студент 4 курса
Группы ИИ-26
Ковальчук А.И.

Проверила:

Андренко К.В.

Цель: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Общее задание:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib).
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1.
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата).
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 7

№ вариант а	Выборка	Размер исходного изображения	Оптимизатор
7	Fashion-MNIST	28X28	Adam

Разработанная модель представляет собой адаптацию свёрточной сети ResNet-18 (He et al., 2016) к задаче классификации изображений набора Fashion MNIST: одноканальные снимки одежды размера 28×28 28×28, десять взаимно исключающих классов. Исходная ImageNet-версия сети рассчитана на трёхканальные изображения 224×224 224×224 и 1000 классов; в работе сохранены остаточная топология и схема четырёх стадий, однако входной тракт, пространственное сжатие и классификатор приведены в соответствие с малым разрешением и числом категорий целевого датасета.

Базовый элемент — остаточный блок с двумя свёртками $3 \times 3 \times 3$. Прямой путь имеет вид

$$\text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(x))), \text{N}(\text{Conv}_{3 \times 3}(\cdot)),$$

после чего к результату добавляется тождественное отображение входа (skip connection) и применяется ReLU. Смещение в свёртках отсутствует: его роль выполняет следующая пакетная нормализация. Если число каналов или пространственный шаг меняются, тождественный путь заменяется проекцией 1×1 с пакетной нормализацией, чтобы размерности слагаемых совпали.

Полный тракт обработки таков.

1. Начальная свёртка. Вместо ImageNet-stem (7×7 , шаг 2 и max-pooling) используется свёртка 3×3 с шагом 1, один входной канал и 64 выходных, затем пакетная нормализация и ReLU. Пространственный размер 28×28 на этом этапе сохраняется.
2. Четыре стадии по два остаточных блока: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ каналов. Первые блоки стадий 2–4 имеют шаг 2 и выполняют даунсэмплинг. Карта признаков последовательно сжимается примерно как $28 \rightarrow 14 \rightarrow 7 \rightarrow 4$.
3. Глобальное среднее пулирование $\text{AdaptiveAvgPool}_{2d}(1,1)$ сворачивает карту в вектор длины 512 независимо от точного пространственного размера.
4. Линейный классификатор отображает 512 признаков в 10 логитов классов.

Веса свёрток инициализируются по схеме Kaiming (He) для ReLU; параметры пакетной нормализации задаются как $\gamma=1$, $\beta=0$. На выходе сеть формирует вектор логитов размера $(N,10)$, согласованный с функцией потерь кросс-энтропии.

Программная реализация:

```
class ResidualBlock(nn.Module):
```

```

    def __init__(self, in_channels, out_channels, stride=1,
downsample=None):

        super().__init__()

        self.conv1 = nn.Conv2d(

            in_channels, out_channels, kernel_size=3,
stride=stride, padding=1, bias=False

        )

        self.bn1 = nn.BatchNorm2d(out_channels)

        self.relu = nn.ReLU(inplace=True)

        self.conv2 = nn.Conv2d(

            out_channels, out_channels, kernel_size=3,
padding=1, bias=False

        )

        self.bn2 = nn.BatchNorm2d(out_channels)

        self.downsample = downsample

def forward(self, x):

    identity = x

    out = self.relu(self.bn1(self.conv1(x)))

    out = self.bn2(self.conv2(out))

    if self.downsample is not None:

        identity = self.downsample(x)

    return self.relu(out + identity)

class ResNet18(nn.Module):

```

```

def __init__(self, num_classes=NUM_CLASSES, in_channels=1):
    super().__init__()

    self.in_channels = 64

    self.conv1 = nn.Conv2d(in_channels, 64, kernel_size=3,
stride=1, padding=1, bias=False)

    self.bn1 = nn.BatchNorm2d(64)

    self.relu = nn.ReLU(inplace=True)

    self.layer1 = self._make_layer(64, 2)

    self.layer2 = self._make_layer(128, 2, stride=2)

    self.layer3 = self._make_layer(256, 2, stride=2)

    self.layer4 = self._make_layer(512, 2, stride=2)

    self.avgpool = nn.AdaptiveAvgPool2d((1, 1))

    self.fc = nn.Linear(512, num_classes)

    self.apply(initialize_weights)

def _make_layer(self, out_channels, blocks, stride=1):
    downsample = None

    if stride != 1 or self.in_channels != out_channels:
        downsample = nn.Sequential(
            nn.Conv2d(self.in_channels, out_channels,
kernel_size=1, stride=stride, bias=False),
            nn.BatchNorm2d(out_channels),
        )


```

```

        layers = [ResidualBlock(self.in_channels, out_channels,
stride, downsample)]

        self.in_channels = out_channels

        for _ in range(1, blocks):

            layers.append(ResidualBlock(out_channels,
out_channels))

        return nn.Sequential(*layers)

    def forward(self, x):

        x = self.relu(self.bn1(self.conv1(x)))

        x = self.layer2(self.layer3(self.layer4(self.layer1(x))))

        x = torch.flatten(self.avgpool(x), 1)

        return self.fc(x)

with torch.no_grad():

    _check = ResNet18()(torch.randn(2, 1, 28, 28))

print("Проверка forward:", tuple(_check.shape))

```

Результаты обучения

Сеть обучалась со следующими установленными гиперпараметрами:

```

IMAGE_SIZE = 28

FASHION_MEAN = 0.2860

```

```
FASHION_STD = 0.3530

BATCH_SIZE = 96

NUM_EPOCHS = 12

LEARNING_RATE = 3e-4

WEIGHT_DECAY = 1e-4

LABEL_SMOOTHING = 0.1

ADAM_BETAS = (0.9, 0.999)

ETA_MIN = 1e-6

VAL_SIZE = 0.1

SEED = 42

NUM_WORKERS = 0

CPU_THREADS = min(8, os.cpu_count() or 8)

USE_FOCAL_LOSS = False
```

Прогресс по эпохам:

Epoch 1/12 - train_loss: 0.9185, train_acc: 0.8115, val_loss: 0.7653, val_acc: 0.8803

Epoch 2/12 - train_loss: 0.7698, train_acc: 0.8777, val_loss: 0.7349, val_acc: 0.8930

Epoch 3/12 - train_loss: 0.7290, train_acc: 0.8963, val_loss: 0.7296, val_acc: 0.8970

Epoch 4/12 - train_loss: 0.7087, train_acc: 0.9056, val_loss: 0.6714, val_acc: 0.9217

Epoch 5/12 - train_loss: 0.6880, train_acc: 0.9158, val_loss: 0.6786, val_acc: 0.9192

Epoch 6/12 - train_loss: 0.6710, train_acc: 0.9226, val_loss: 0.6634, val_acc: 0.9247

Epoch 7/12 - train_loss: 0.6579, train_acc: 0.9286, val_loss: 0.6542, val_acc: 0.9318

Epoch 8/12 - train_loss: 0.6446, train_acc: 0.9346, val_loss: 0.6582, val_acc: 0.9277

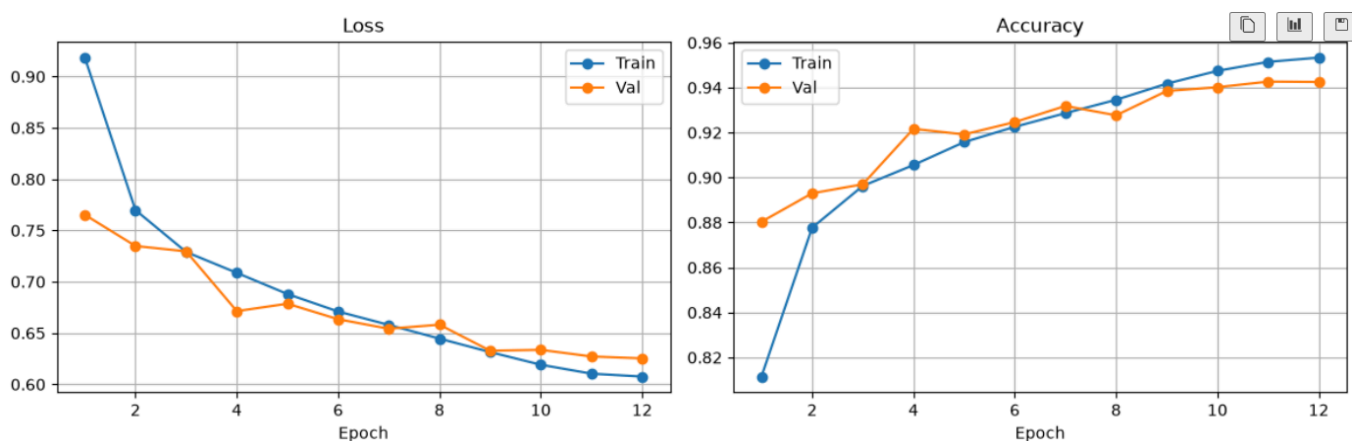
Epoch 9/12 - train_loss: 0.6315, train_acc: 0.9417, val_loss: 0.6328, val_acc: 0.9385

Epoch 10/12 - train_loss: 0.6191, train_acc: 0.9474, val_loss: 0.6336, val_acc: 0.9402

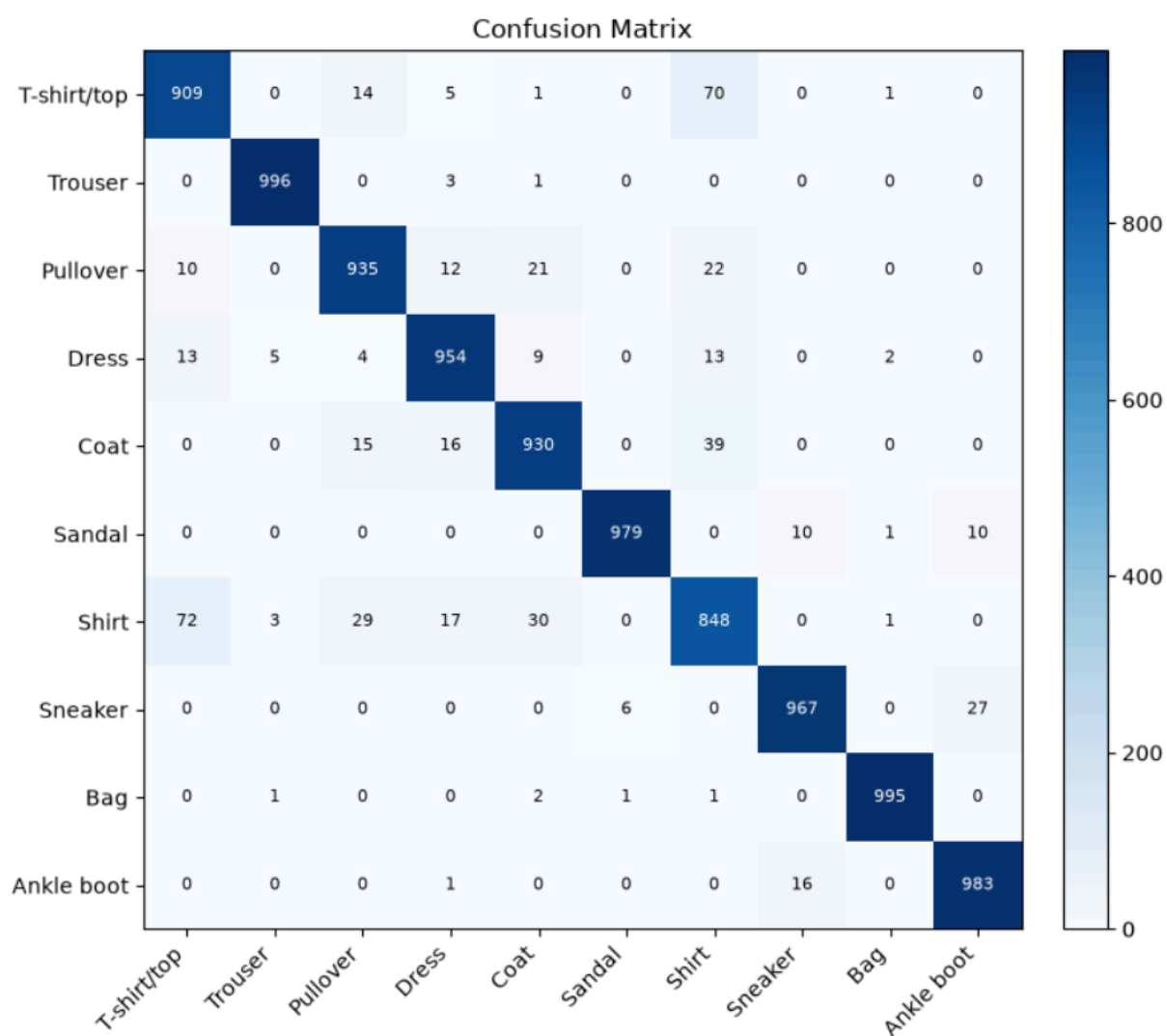
Epoch 11/12 - train_loss: 0.6104, train_acc: 0.9514, val_loss: 0.6272, val_acc: 0.9427

Epoch 12/12 - train_loss: 0.6076, train_acc: 0.9533, val_loss: 0.6253, val_acc: 0.9425

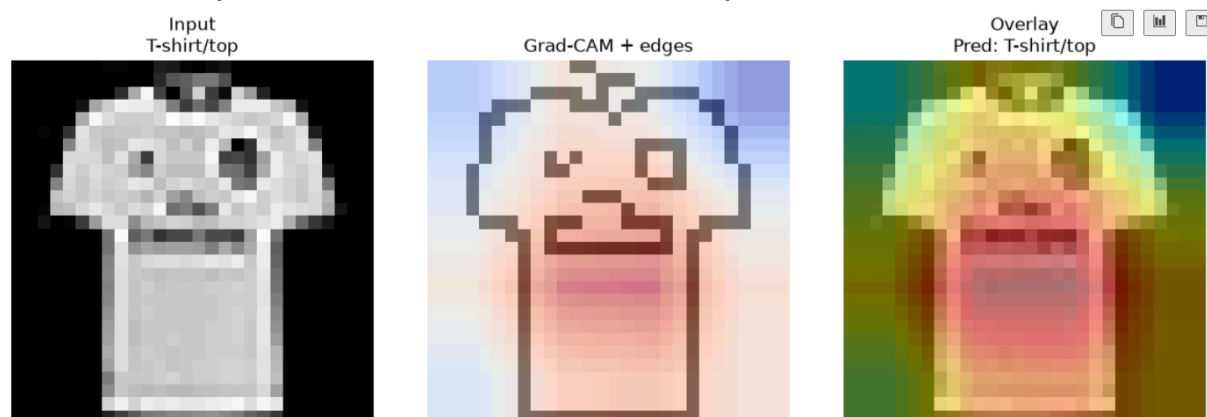
Графики изменения функции потерь и значения точности на обучающем и валидационных наборах данных:



Матрица ошибки:



Тепловая карта активаций выходного свёрточного слоя:



Вывод. Остаточные связи облегчают обучение сравнительно глубокой сети: градиент проходит по короткому тождественному пути, что снижает риск деградации точности при увеличении глубины. Для Fashion MNIST это существенно, поскольку классы визуально близки (футболка, рубашка, пальто, пуловер) и требуют иерархии признаков – от краёв и текстуры ткани до силуэта изделия.

Свёрточная иерархия с постепенным даунсэмплингом сочетает локальные фильтры малого рецептивного поля с расширением каналов. На малых изображениях одежды это позволяет выделить как мелкие детали (швы, каблук, ручки сумки), так и глобальную форму, не прибегая к полносвязной сети на «сырых» пикселях.

Адаптация stem под 28×28 критична. Исходный ImageNet-вход с шагом 2 и max-pooling слишком рано уничтожает пространственную информацию: после нескольких стадий карта вырождается почти в одну ячейку. Свёртка 3×3 с шагом 1 сохраняет разрешение на старте, и даунсэмплинг выполняется только там, где это нужно для построения более абстрактных признаков.